

M5 – Database Management

PROGRAM	ASSESSMENT CODE	TOTAL TIME	STACK
Software Engineering	M5-A1	3 Hours	SQL

GENERAL INSTRUCTIONS

Attempt all sections. Write all queries and PL/SQL blocks in standard SQL compatible with MySQL 8.x unless stated otherwise. For coding tasks, submit your .sql source file along with a screenshot of your query output from MySQL Workbench or CLI. Label every task clearly with its section and task number. All work must be your own.

A Section A — Concept Application

Answer all questions. Each question presents a real-world scenario followed by a concept-specific question.

SCENARIO

S
1

You are designing the database for a food delivery application. It stores restaurant details, menu items linked to each restaurant, and customer orders. You must decide which columns need NOT NULL constraints, which serve as PRIMARY KEYS, and which link tables via FOREIGN KEYS.

Question: Justify your constraint choices for at least three columns across two tables. What data integrity problems would arise if these constraints were not applied?

SCENARIO

S
2

You are managing the database for a food delivery platform. The operations team asks you to update the status of all orders placed more than 7 days ago that are still marked 'Pending' to 'Cancelled', and to permanently delete menu items marked 'Unavailable' from restaurants that have closed.

Question: Explain why the WHERE clause is essential in each DML statement. What could go wrong with an UPDATE or DELETE executed without a proper WHERE condition?

SCENARIO

S
3

You are building a revenue dashboard for a food delivery startup. The management wants total revenue, average order value, and number of orders per restaurant — but only for restaurants that have processed more than 20 orders in the current month.

Question: Which SQL clauses and aggregate functions would you use to produce this report? Explain why HAVING must be used here rather than WHERE to filter by aggregate results.

SCENARIO

S
4

You are developing an order history page. The orders table has order_id, customer_id, restaurant_id, and total_amount. The restaurants table has restaurant_id and name. The page must show order ID, restaurant name, and amount for every order — including any that reference a restaurant no longer in the table.

Question: Compare INNER JOIN and LEFT JOIN for this query. Which would you use and why? Describe what each join type does to records that have no match in the other table.

SCENARIO

S
5

You are building the analytics layer for a food delivery platform. A senior developer proposes a VIEW called top_restaurants showing restaurant name, total orders, and average rating — hiding raw ID columns from the front-end team.

Question: Justify why a VIEW is preferred over direct table access here. What are two limitations the front-end developer should be aware of when working with this VIEW?

SCENARIO

S
6

You are implementing the payment module for a food delivery app. When a customer places an order, the system must deduct the amount from the customer's wallet balance and record the payment simultaneously. During load testing, the wallet deduction completes but the payment record insert occasionally fails, leaving balances inconsistent.

Question: Which database mechanism should have prevented this inconsistency? Explain how COMMIT, ROLLBACK, and SAVEPOINT work together to ensure both operations succeed or neither takes effect.

B

Section B — Practical Coding Tasks

Complete all tasks. Each task targets one or more specific SQL/PL/SQL concepts from the module. Submit your .sql file and a screenshot of the output from MySQL Workbench or CLI.

Task 1: Design and Populate a Restaurant Database

Create a structured schema for a food delivery system and populate it with sample data using DDL and DML commands.

- Create a database named foodapp_db and a table named restaurants with columns: restaurant_id (INT, AUTO_INCREMENT, PRIMARY KEY), name (VARCHAR, NOT NULL), city (VARCHAR), cuisine_type (VARCHAR), and rating (DECIMAL(3,1)).
- Insert at least 6 restaurant records spanning at least 3 different cities.
- Update the rating of one restaurant and delete one restaurant record using a WHERE condition.
- Write a SELECT query that retrieves all restaurants sorted by rating in descending order.

Task 2: Aggregate Revenue Reporting on Orders

Write SELECT queries using aggregate functions and GROUP BY to produce a restaurant-level revenue summary from order data.

- Create an orders table with columns: order_id, restaurant_id, customer_name, total_amount, and order_date. Insert at least 12 orders distributed across at least 4 restaurants.
- Write a query to show total revenue and average order value per restaurant.
- Write a query using HAVING to list only restaurants whose total revenue exceeds Rs 5,000.
- Write a query to retrieve the 5 most recent orders using ORDER BY and LIMIT.

Task 3: Multi-Table JOINS and Menu Summary View

Build a two-table schema for restaurants and menu items, write JOIN queries to combine data, and create a summary VIEW.

- Create two tables: restaurants (restaurant_id, name, city) and menu_items (item_id, restaurant_id, item_name, price, category). Link them with a FOREIGN KEY on restaurant_id.
- Insert at least 5 restaurants and 12 menu items; at least one restaurant must have no menu items.
- Write an INNER JOIN query listing restaurant name, item name, and price for all available items.
- Write a LEFT JOIN query so that restaurants with no items also appear in results, with NULL values in item columns.
- Create a VIEW named restaurant_menu_summary showing restaurant name, total number of menu items, and average item price grouped by restaurant.

Task 4: PL/SQL Order Placement Procedure

Write a stored procedure to process a customer order with wallet deduction and full transaction control.

- Create a stored procedure named `place_order` that accepts `customer_id`, `restaurant_id`, and `amount` as parameters.
- Deduct the amount from the customer's `wallet_balance` in a `customers` table and insert a new record into the `orders` table.
- Use a transaction block: if the customer's `wallet_balance` is insufficient, `ROLLBACK` everything; otherwise `COMMIT`.
- Test with a valid order and an order exceeding wallet balance. Screenshot both outcomes from MySQL Workbench.

C

Section C — Mini Capstone Project

Mini Project: Food Delivery Platform — Backend Database

Objective:

Build the complete backend database for a Food Delivery application, combining DDL, DML, JOINS, Views, and PL/SQL into a single integrated .sql script.

Your project must:

- Create three tables: `restaurants` (`restaurant_id`, `name`, `city`, `cuisine_type`, `created_at`), `menu_items` (`item_id`, `restaurant_id`, `item_name`, `price`, `category`) with a FOREIGN KEY on `restaurant_id`, and `orders` (`order_id`, `customer_name`, `restaurant_id`, `item_id`, `quantity`, `total_amount`, `order_date`) with a FOREIGN KEY on `restaurant_id` — all with appropriate PRIMARY KEY and NOT NULL constraints.
- Populate with at least 3 restaurants, 4 menu categories, and 15 order records distributed across restaurants.
- Create a VIEW named `restaurant_sales_summary` showing restaurant name, total number of orders, and total revenue grouped by restaurant.
- Write a stored procedure named `add_order` that accepts `customer_name`, `restaurant_id`, `item_id`, and `quantity`, calculates `total_amount` from the menu item price, validates the restaurant exists (`ROLLBACK` if not), and inserts the order record.
- Write a TRIGGER named `after_order_insert` that logs a record into an `order_audit` table (`order_id`, `restaurant_id`, `action='INSERT'`, `log_time=NOW()`) immediately after every new order is created.

D

Section D — AI-Augmented Learning

STEP 1 • BUILD WITH AI

Use an AI tool of your choice (ChatGPT, Claude, GitHub Copilot, etc.) to help you write a stored procedure that:

- Accepts a `restaurant_id` and a month number (1–12) as inputs and returns a summary showing total orders, total revenue, and average order value for that restaurant in that month.
- Handles the case where the restaurant has no orders in that month by returning a meaningful message rather than an empty result.
- Uses a `SELECT` with `GROUP BY` and aggregate functions to compute the summary values correctly.
- Runs successfully in MySQL Workbench — screenshot output for at least two different `restaurant_id` and month combinations.

STEP 2 • TEST & DEBUG (WITHOUT AI)

Then, working without AI, test the stored procedure and find at least one bug, limitation, or improvement in the AI's solution. Fix it yourself.

SUBMIT

All 3 items required

- 1 The exact prompt(s) you gave the AI tool.
- 2 The AI's original stored procedure code and your corrected/improved version.
- 3 A 3–4 line note explaining what you changed and why the AI's version needed improvement.